【扔物线 Compose 课程讲义】3.3-重组作用域和 remember()

扔物线学堂 - 秉心 (1437182)

Recompose Scope（重组作用域）

remember()

关于讲义

讲义是对课程内容中**所有结论的集中汇总**。所以在看完每节课程之后，迅速来看一遍讲义，对课程内容做回顾，会有非常好的吸收效果。

面试前不要背讲义

在面试之前，通过快速浏览一遍讲义来确认所有知识都没有忘记，是个很好的备战方式。

但**不要用【背讲义上的知识点】的方式来备战**。最有效的浏览方式是：

浏览讲义中的所有内容，并：

- 看到「懂，并且记得背后的原因（或技术细节）」的，直接跳过；
- 看到「懂，但细节已经忘了」的，快速回看当节课程，把知识捡起来；
- 看到「已经不太懂 / 完全忘了」的，认真重看当街课程，把知识学回来。

疑问或意见

如果你发现讲义或课程有错误，或有别的什么意见，可以向任何一个你认识的助教反馈。

如果认可我们的课程，还请给个好评！ 评价链接：rengwuxian.com/cp

Recompose Scope（重组作用域）

指的是在 Compose 的 `@Composable` 代码中，被划归为一体的代码块，这些代码块会在变量发生变化从而导致界面需要重新组合（重组、Recompose）的时候，被一起标记为无效、并在稍后一起执行。

具体的 Recompose Scope 细节，还得重看视频。

`remember()`

- 由于 Recompose Scope（重组作用域）的机制，变量的初始化代码可能会被「捎带着」发生重复执行，从而导致程序逻辑出错；
  - 使用 `remember() {}` 来包住变量的初始化代码，可以确保变量只被初始化一次，哪怕初始化部分的代码被重复执行；
- `remember()` 函数也可以有一个或多个参数作为 key 存在，在 key 变化的时候就重复初始化，来应对按需重新初始化的场景；
- 由于 `remember()` 应对的就是 Recompose 导致的变量重复初始化的问题，所以在 Composable 函数之外没必要使用 `remember()`，也不能使用 `remember()`（因为它是个 Composable 函数）。